

CS-3083 Project 3 Report

Task 1 Report

Propositional Logic Entailment

Function Design

Entailment was determined by checking whether the conjunction $KB \wedge \neg \alpha$ is unsatisfiable. The knowledge base and the query were represented as sets of clauses, and each clause was represented as a set of literals. Helper functions were used to parse clauses, negate the query, detect complementary literals, resolve two clauses, check for tautologies, and evaluate whether a clause is satisfied under a partial model.

The PL-Resolution algorithm was implemented by adding the negation of the query to the knowledge base and repeatedly generating resolvents. If the empty clause is produced, then the knowledge base entails the query. To make the comparison stable in Jupyter, the updated version places a maximum time limit and a maximum clause-growth limit on the benchmark runs for PL-Resolution. This prevents the notebook from appearing to loop forever on difficult random instances.

The DPLL algorithm was implemented as a recursive backtracking search over truth assignments. The implementation checks whether clauses are satisfied, unsatisfied, or undecided under the current model. It also uses simplification steps before branching, including pure symbol detection and unit clause propagation. If the clause set becomes unsatisfiable, DPLL returns false for satisfiability, which means the original entailment is true.

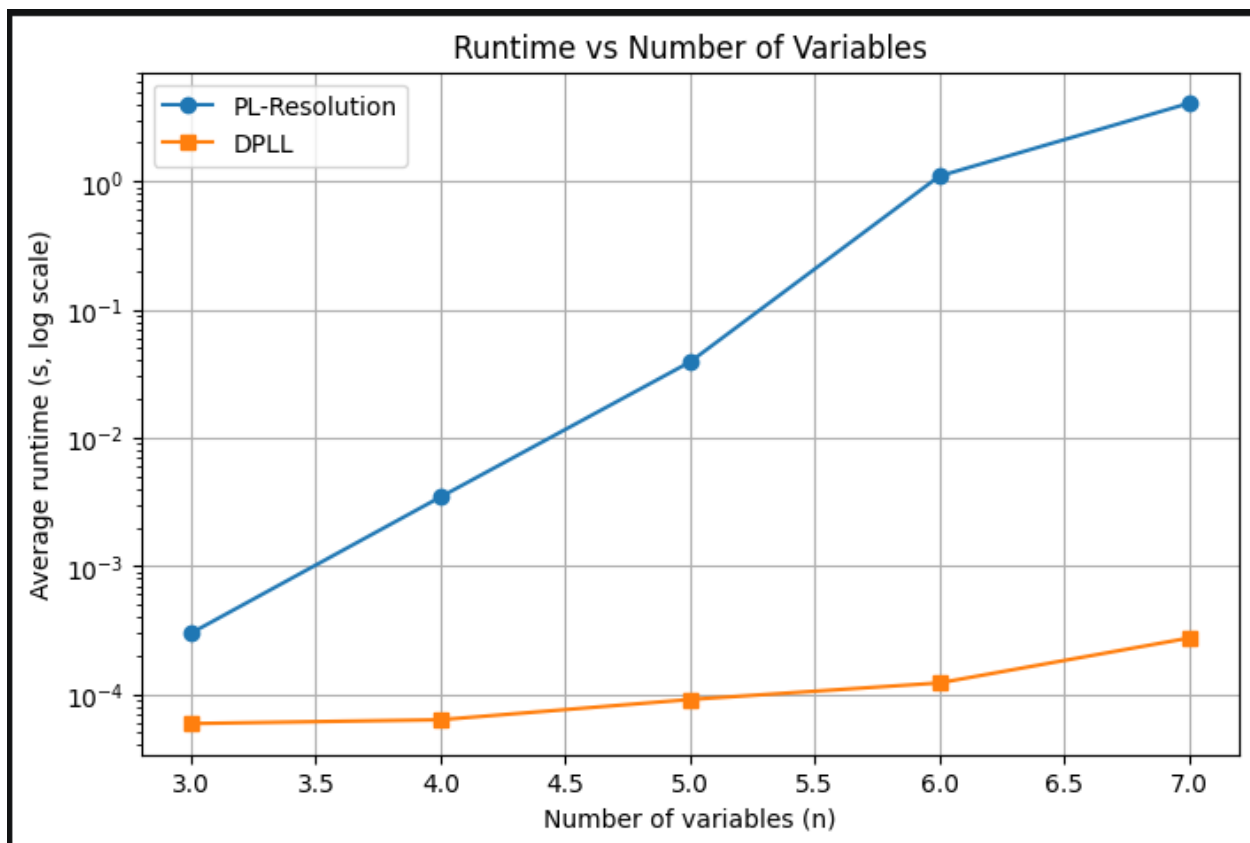
Experimental Setup

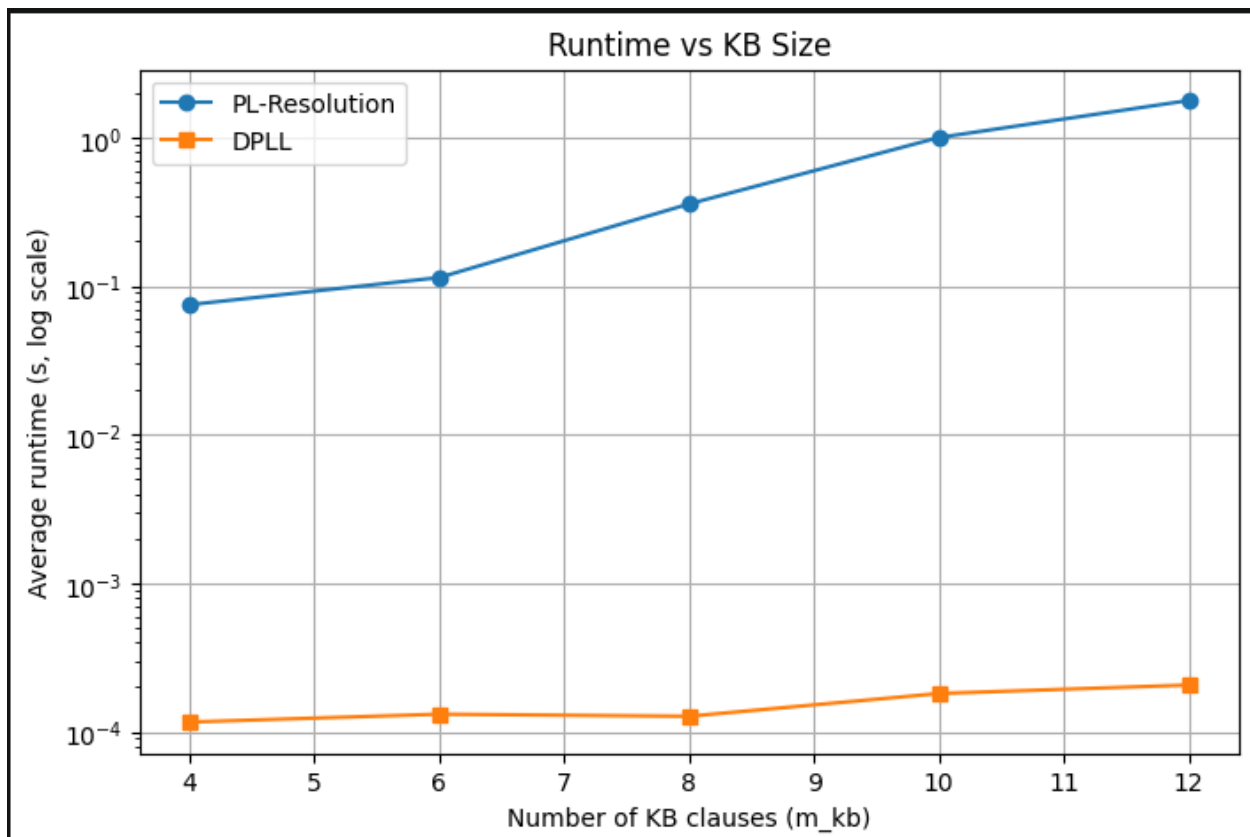
Three types of tests were used. First, manually defined knowledge bases and queries were tested to verify correctness on known examples. Second, randomly generated knowledge bases and queries were used to compare the algorithms on a wider range of inputs. Third, a bounded comparison function was used to measure average running time under two conditions: increasing the number of variables and increasing the number of clauses in the knowledge base.

The comparison function was upgraded so that it does not keep running indefinitely in Jupyter. The benchmark now uses small, controlled problem sizes, repeats each experiment a limited number of times, and applies a timeout to PL-Resolution benchmark runs. If a run reaches the timeout, the function records the capped runtime and continues instead of hanging the notebook.

Results

Both algorithms returned the same entailment result on the correctness tests that were run. This indicates that the implementations are consistent with each other on the tested inputs. The comparison benchmark then measured average execution time for both algorithms.





Running Time

The updated comparison results show that DPLL remained consistently faster than PL-Resolution on the tested inputs. As the number of variables increased, PL-Resolution took longer because it had to generate and compare many possible resolvents. DPLL increased more gradually because it searched assignments more directly and simplified the problem before branching.

When the number of clauses in the knowledge base increased, the same trend was observed. PL-Resolution became slower much more quickly, while DPLL remained comparatively efficient. The timeout safeguard was not added to improve PL-Resolution itself; it was added so that the comparison function would remain usable inside a notebook environment and complete in a predictable amount of time.

Completeness

Both PL-Resolution and DPLL are complete algorithms for propositional logic. PL-Resolution is complete because if the knowledge base entails the query, repeated application of the resolution rule will eventually derive the empty clause from $KB \wedge \neg \alpha$. DPLL is complete because it systematically explores the possible truth assignments for the symbols and determines whether the clause set is satisfiable or unsatisfiable.

The benchmark safeguards do not change the theoretical completeness of the algorithms themselves. They only limit how long the benchmark waits during experimental comparison. In other words, the algorithm remains complete in theory, but the benchmark is intentionally bounded so the notebook can finish running reliably.

Scalability

The runtime plots indicate that PL-Resolution scales less effectively than DPLL on these experiments. As the propositional logic problem grows, the number of possible resolvents can increase rapidly, which causes PL-Resolution to slow down. DPLL also has exponential worst-case behavior, but its simplification steps allow it to handle larger instances more efficiently in practice.

The upgraded comparison function improves experimental scalability in the notebook because it prevents a single hard random instance from dominating the entire run. This makes the benchmarking process much more practical and repeatable.

Analysis of Changes

The main code change was not just adding more plots. The main improvement was making the comparison function bounded and predictable. The benchmark now uses smaller controlled ranges, a fixed random seed for reproducibility, a limited number of repeats, and a timeout for PL-Resolution. This makes the notebook finish cleanly instead of appearing stuck.

These changes make the project stronger because the comparison section now produces useful evidence without risking an endless run in Jupyter. The results can still be discussed in terms of running time, completeness, and scalability, which matches the assignment requirements.

Figure Captions

Figure 1. Running time comparison of PL-Resolution and DPLL as the number of variables increases.

Figure 2. Running time comparison of PL-Resolution and DPLL as the number of clauses in the knowledge base increases.